WEEKDAY

From a spreadsheet queue to a scheduling control tower

Problem statement, solution rationale, edge cases, assumptions and test plan

My approach I treated this as an operations problem first and a software problem second. The goal was to make work visible, make the next action clear, and reduce the chance that a request quietly gets lost between a spreadsheet, an intern and a candidate.

Prepared as an assignment submission

Product thinking and workflow design: my own analysis and decisions

Application implementation: AI-assisted / vibe-coded prototype built with Google AI Studio and a React + TypeScript stack

This document distinguishes the intended operating model from what the current prototype actually implements. Integrations described as simulated are not represented as live production connections.

The question: click_here *(the assignmnet scenario document)*
The project link: click_here *(redirects to Github)*
The deployed application: click_here (*redirects to Vercel deployment)*

Table Of Contents

Table Of Contents 1
1. The task I was solving 2
What success should look like 3
2. Breaking the problem into smaller units 3
The central design principle 4
3. The solution I carried forward 4
What is included in the prototype 4
The intended workflow 4
4. Why I chose this solution 4
What I took from my HR Analyst experience at Chakradhara 5
Options I considered 5
Why not automate everything immediately? 5
5. Assumptions and decisions I made 6
Additional assumptions I would validate 6
6. Edge cases I considered 7
7. Test cases and expected outcomes 8
8. How I would measure whether it works 9
9. How I built it: my role, AI assistance and technology 10
Technology used in the application 10
Current limitations I would disclose 10
10. What I would do next 11
11. Closing note 11

1. The task I was solving

The assignment is to design a reliable way to manage a high-volume interview-scheduling operation. The working brief I used was:

> **Target** Handle 500+ scheduling requests per day and achieve at least 80% booking within 24 hours of a request being created.

The operation has two interns, Ram and Shyam, who need to work through requests consistently. A manager needs to understand what is moving, what is stuck, what is close to breaching the SLA, and where intervention is required. The source workbook, "Scheduling ToDos.xlsx", is the starting point.

I interpreted the task as more than sending a Calendly link. A request can be contacted without being booked; it can have a broken link, no available slots, an unresponsive candidate, a duplicate row, or a timestamp that is already old when the file is imported. Those differences matter if the team is expected to report an honest booking rate.

What success should look like

- Every valid request has a clear owner, current status, next action and deadline.
- Interns can prioritize the work that needs attention instead of scanning a large sheet row by row.
- Managers can see the queue, SLA risk and exceptions without manually combining multiple tabs.
- A booking is counted only when there is evidence of a booking not simply because outreach was sent.
- The team can explain why a request is delayed and what was done about it.

2. Breaking the problem into smaller units

I broke the brief into work units before deciding what to build. This kept the scope grounded in the day-to-day work and made it easier to identify where errors could enter.

Work unit	Question to answer	What the solution needs to do
1. Intake and data quality	Can we trust the rows coming from the workbook?	Read supported sheets, validate required fields, preserve source details and report rejected or duplicate rows.
2. Ownership and workload	Who is responsible for each request?	Assign work consistently between Ram and Shyam and retain an audit trail if ownership changes.
3. Status and next action	What has happened, and what should happen next?	Use defined statuses, log outreach and make follow-ups visible.
4. SLA and prioritisation	Which request needs attention first?	Calculate time remaining from the original creation time and surface risk before the deadline.
5. Booking verification	Is the interview actually booked?	Require booking evidence before treating the request as complete.
6. Exceptions	What cannot be solved by normal outreach?	Flag broken links, unavailable slots, withdrawals, cancellations and other blockers for human resolution.
7. Manager visibility	Is the operation meeting the target?	Show queue volume, status mix, SLA performance and overdue work in one place.

The central design principle

I wanted one request to have one understandable story: where it came from, who owns it, what has been tried, what is due next, how much SLA time remains, and how the final outcome was verified. If a manager has to reconcile three sheets and ask two people for updates to answer those questions, the process is still too fragile.

3. The solution I carried forward

I built a prototype called the Weekday Scheduling Control Tower. It is an operations workspace around the scheduling queue, rather than another static tracker. The main idea is to bring the intern workflow, manager view and operating rules into one application.

What is included in the prototype

Area	Purpose	How it helps
Management Control Tower	Summary of the queue and SLA position.	Lets a manager spot risk and exceptions quickly.
Central Tracker	Searchable view of scheduling requests and their statuses.	Creates a single working view instead of relying on manual sheet filters.
Ram's Workbench	Task-focused queue for Ram.	Shows assigned work and the next actions to take.
Shyam's Workbench	Task-focused queue for Shyam.	Makes ownership and workload separation explicit.
Request detail and action flows	Record outreach, follow-ups, blockers, reassignment and outcomes.	Keeps a history of operational actions rather than relying on memory.
SLA and follow-up logic	Calculate time remaining and highlight urgency.	Encourages action before a request becomes overdue.
Demo Center and virtual clock	Walk through normal and exception scenarios without waiting a full day.	Makes the logic easier to demonstrate and test in an interview.
SOP and test plan views	Expose operating rules and test scenarios in the application.	Helps the team follow the same process and understand edge cases.

The intended workflow

1. Import the scheduling workbook and review the import summary.
2. Validate the candidate and interview details; identify duplicate or incomplete rows.
3. Assign each valid request to Ram or Shyam.
4. Prioritise by SLA risk and the next action due.
5. Log outreach and schedule follow-ups rather than leaving reminders in someone's head.
6. Escalate blockers such as invalid links or unavailable slots.
7. Mark a request booked only after booking evidence is available.
8. Review the manager dashboard and use the audit trail to understand exceptions.

4. Why I chose this solution

My choice came from seeing how operational data is handled in real teams, not from wanting to build an application for its own sake.

What I took from my HR Analyst experience at Chakradhara

During my HR Analyst internship at Chakradhara Aerospace & Cargo, I worked around HR reporting and employee-related data. In that kind of work, spreadsheets are useful and often the quickest way to get started. The trouble begins when a sheet becomes the process itself: people add their own columns, status labels are interpreted differently, a follow-up is recorded in one place but not another, and the latest version is not always obvious. Even when every person is trying to do the right thing, maintaining consistency takes effort.

That experience shaped how I looked at this scheduling task. With hundreds of requests arriving each day, the risk is not only that a person makes a mistake. The process can make mistakes easy: duplicated rows, stale statuses, unclear ownership, missed reminders, inconsistent timestamps and manual reporting all create opportunities for work to slip through.

I did not assume the team was using spreadsheets badly. I assumed the operation needed a system that would make the correct action easier and make exceptions visible. A spreadsheet remains useful for intake and export, but it should not be the only place where ownership, deadlines, follow-ups and booking evidence live.

Options I considered

Option	Strength	Why I did / did not choose it
Continue with the workbook only	Fast to start; familiar to everyone.	It is still useful as an input/output format, but manual ownership, follow-ups, deduplication and SLA monitoring become difficult to maintain at higher volume.
Add more spreadsheet formulas and colour coding	Low implementation effort; familiar workflow.	Could improve visibility, but rules and actions would still be spread across tabs and depend on people updating the sheet correctly.
Build a scheduling control tower prototype	Combines queue visibility, task ownership, SLA rules and exception handling.	I chose this because it addresses the workflow end to end while retaining the workbook as an import source.

The key product decision I chose to make the next action and the exception visible not just display another status column. The aim is to reduce the amount of manual coordination required to keep the process moving.

Why not automate everything immediately?

I kept the prototype focused on the workflow and its decision rules first. Real email, phone and Calendly integrations require credentials, API access, error handling, privacy controls and operational approval. Simulating those actions is useful for demonstrating the flow, but it is not the same as claiming a live integration. I would validate the workflow with users before investing in those connections.

5. Assumptions and decisions I made

The assignment leaves some operational details open. I made the following assumptions so I could build a coherent prototype and test it. These should be confirmed with the operations owner before a production rollout.

Assumption / decision	Why I used it	What should be confirmed
The SLA starts at the original request creation time.	Importing a late request must not make it look new or reset the clock.	Which source timestamp is authoritative, including timezone and missing values
The target is 80% of eligible requests booked within 24 hours.	This turns the brief into a measurable operational outcome.	How are eligible, excluded, cancelled and withdrawn requests defined in the official KPI?
A booking requires verifiable evidence.	Sending a message is not the same as a completed booking.	Will the source of truth be a Calendly webhook, calendar event, or approved manual verification?
Ram and Shyam share the initial workload evenly.	A predictable split is simple to explain and test.	Should allocation later consider capacity, shift, skill or partner account?
Even source-row indexes go to Ram and odd indexes to Shyam in the prototype.	It provides deterministic assignment for a repeatable demo.	A production allocator should balance active workload, not just row order.
Matching candidate email + company + interview round indicates a duplicate.	The same candidate can legitimately have multiple interview rounds.	Confirm the unique key and how to handle changed email addresses or rescheduled rounds.
SLA bands are green above 12 hours, amber from 4 to 12 hours, red below 4 hours, and overdue after the deadline.	A simple visual threshold helps the team prioritise.	Confirm whether these thresholds match actual staffing and escalation coverage.
The first follow-up is due four hours after initial outreach.	It provides a concrete default for the prototype.	Confirm candidate communication policy, local time windows, weekends and holidays.
The virtual clock is for demonstration and testing.	A live 24-hour lifecycle cannot be shown in a short review.	Never use virtual time for real operational reporting.
The workbook is an intake source, not a permanent source of truth.	The goal is to reduce manual reconciliation and protect workflow history.	Agree which system owns the final record and how exports are reconciled.

Additional assumptions I would validate

- The workbook contains a dependable request-created timestamp and enough identifiers to match records.
- A request may have multiple interview rounds, but each individual round should have its own status and SLA outcome.
- Candidate withdrawal and interview cancellation are different outcomes and should not be reported as successful bookings.
- Requests already older than 24 hours at import should immediately appear as overdue if still eligible and unbooked.
- A request should not receive repeated reminders after it is booked, withdrawn or explicitly closed.
- Reassignment must preserve the original deadline and leave an audit note.
- The application’s demo data is fictional; real candidate information should not be used in a public demo.

6. Edge cases I considered

The normal path import, send link, candidate books is the easiest part. I focused on the cases that can make a queue unreliable or a KPI misleading.

Edge case	Expected handling / why it matters
Missing candidate name or email	Reject or quarantine the row with a clear validation reason; do not silently create an unusable task.
Malformed email address	Flag for correction before outreach; do not assume the message was delivered.
Missing company, interview round or request ID	Validate required fields and use a safe fallback only where an agreed rule exists.
Missing or unparseable creation timestamp	Flag the row; do not quietly start a new 24-hour SLA at import time.
Timestamp in the future or wrong timezone	Flag as suspicious and normalise timestamps consistently before calculating SLA.
Same workbook imported twice	Detect duplicates and report how many were skipped; avoid creating duplicate work.
Same candidate, company and round repeated	Treat as a likely duplicate unless an authorised reason indicates otherwise.
Same candidate and company but different interview round	Keep as a separate valid request with its own lifecycle.
Candidate email changed	Avoid relying on email alone for every historical match; provide a review path for identity changes.
Broken or malformed scheduling link	Flag a blocker and prevent routine outreach until a valid link is supplied.
No available slots	Pause the normal follow-up loop and route to the scheduling owner or partner contact.
Candidate does not respond	Surface the follow-up when due; keep the request open and escalate based on the SLA.
Candidate replies that they are not interested	Record withdrawal, stop future reminders and exclude it from successful bookings.
Candidate asks to reschedule	Reopen or create the appropriate rescheduling state while preserving the history of the original booking.
Candidate cancels after booking	Do not leave the request permanently marked as successfully completed; record cancellation and define whether it re-enters the queue.
Booking confirmation event arrives twice	Handle repeated events idempotently so one booking does not create duplicate records.
Booking event arrives after a request was withdrawn	Flag the conflict for review instead of blindly overwriting the latest state.
Outreach sent but booking not confirmed	Keep the request unbooked; contact attempts are not booking evidence.
Request is already 30 hours old when imported	Show it as overdue immediately if it is still eligible and unbooked.
Request reassigned between interns	Keep the original created time and deadline; record previous owner, new owner, time and reason.
Intern is absent or has too many active tasks	Escalate or rebalance based on capacity; simple even/odd assignment is only a prototype rule.
Follow-up due at night, weekend or holiday	Use an agreed contact-hours calendar rather than sending at an inappropriate time.
Email or calendar integration fails	Retry safely, log the failure and surface a manual action; do not mark the request complete.
Spreadsheet has unexpected headers, extra sheets or blank rows	Validate the workbook structure and provide a readable import summary instead of failing silently.
Large workbook / 500+ requests per day	Test import speed, rendering, search and filtering at realistic volume; avoid assuming a small demo proves scale.
Two users update the same request	A production version needs server-side concurrency control and an audit trail to prevent lost updates.
Local browser data is cleared or	Current prototype data may not follow the user; production needs a shared database

opened on another device	and backup strategy.
Manager dashboard has no eligible/mature requests	Show N/A rather than 0% when the denominator is zero.
A booking happens just before or after the 24-hour cutoff	Use a consistent timestamp and timezone rule; define whether the event timestamp or ingestion timestamp determines compliance.
A closed or excluded request changes status later	Keep a status history and define whether reopening starts a new SLA or preserves the original one.

7. Test cases and expected outcomes

I organized testing around the risks in the workflow rather than testing only whether screens load. The repository includes an in-app test runner and a documented set of 19 scenarios. The table below summarizes the core cases and the expected behavior.

ID	Test	Setup	Expected outcome
TC-01	Import valid workbook	Valid multi-sheet workbook with required fields.	Rows are parsed, source information is retained and workload is allocated consistently.
TC-02	Import same workbook again	Records from the first import already exist.	No duplicate requests are created; skipped duplicates are reported.
TC-03	Required fields missing	A row has no candidate name or email.	The row is rejected or flagged with a specific reason.
TC-04	Invalid scheduling URL	Request contains a malformed link.	A blocker is visible and routine outreach is not treated as ready.
TC-05	Log initial outreach	A valid uncontacted request is selected.	Outreach attempt and timestamp are recorded; next follow-up becomes visible.
TC-06	Candidate does not respond	Outreach has been logged and follow-up time passes.	Request appears in the due follow-up queue.
TC-07	Advance time to follow-up	Follow-up is scheduled for a future time.	The request becomes due at the expected point in the virtual clock.
TC-08	Approaching SLA deadline	Less than four hours remain and request is unbooked.	Request is marked red / urgent and prioritised.
TC-09	SLA deadline passed	Request remains unbooked after 24 hours.	Request is marked overdue and visible to the manager.
TC-10	Verified booking	A booking confirmation is supplied through the demo flow.	Status becomes booked and pending follow-ups are cancelled.
TC-11	Booking cancelled	A previously booked interview is cancelled.	Cancellation is recorded and the request is reopened or routed for rescheduling according to policy.
TC-12	Duplicate booking event	The same confirmation is received twice.	No duplicate booking or request is created.
TC-13	Candidate withdraws	Candidate explicitly opts out.	Outreach stops and withdrawal reason is recorded.
TC-14	No slots / broken link	The normal booking path cannot proceed.	A blocker is raised and routed for resolution.
TC-15	Reassign between interns	Request is transferred with a reason.	Owner changes, audit note is stored and original SLA deadline is preserved.
TC-16	Import already-overdue request	Creation time is older than 24 hours.	It is immediately classified as overdue if still eligible and unbooked.
TC-17	KPI with no eligible requests	No mature requests are available for the metric.	Dashboard displays N/A rather than a misleading 0%.
TC-18	End-to-end successful	Clean request selected in Demo	Outreach, waiting and booking confirmation

	flow	Center.	produce a complete audit trail.
TC-19	End-to-end escalation flow	Clean request selected; time is advanced.	Follow-up and SLA escalation states appear as expected.

8. How I would measure whether it works

Metric	Working definition	What it tells us
24-hour booking compliance	Eligible requests booked within 24 hours ÷ eligible requests whose 24-hour window has matured.	Whether the operation is meeting the 80% target.
Open requests by SLA band	Count of open eligible requests in green, amber, red and overdue states.	Where the team needs to focus now.
Follow-ups overdue	Open requests whose next action time has passed.	Whether the follow-up process is being maintained.
Exception rate	Requests blocked by invalid links, no slots or other exceptions ÷ total valid requests.	How much work is outside the normal path.
Duplicate rate	Duplicate rows skipped ÷ total rows evaluated.	How much repeated intake is entering the workflow.
Time to first outreach	Time between request creation and first logged outreach.	How quickly the team starts working a request.
Workload by owner	Open requests, due follow-ups and at-risk requests per intern.	Whether the split is actually balanced.
Booking cancellation / reopen rate	Previously booked requests that are cancelled or reopened.	Whether the first booking outcome remains stable.

I would avoid reporting a single overall percentage without showing the denominator and exclusions. For example, withdrawn requests and duplicate rows should not be silently mixed into the same booking-compliance calculation. The operations owner should sign off on the final metric definitions.

9. How I built it: my role, AI assistance and technology

The product idea, problem breakdown, workflow choices, assumptions and edge-case thinking were mine. I decided what problem to solve, what the users needed to see, which rules the prototype should demonstrate and which cases needed to be tested. I used AI-assisted development (often called vibe coding) through Google AI Studio to accelerate the implementation and iteration of the application.

I want to be clear about that distinction: I am not claiming that I manually wrote every line of code from scratch. I used AI as a build partner, while taking responsibility for the product direction, reviewing the generated implementation, shaping the workflows and documenting where the prototype is and is not production-ready.

Technology used in the application

Layer	Technology found in the project	Role
AI-assisted development	Google AI Studio	Used to support application generation and iteration.
Frontend	React 19, TypeScript, Vite	Builds the interactive single-page application and its typed UI logic.
Styling	Tailwind CSS v4, @tailwindcss/vite	Provides the styling system and Vite integration.
Icons and motion	lucide-react, motion	Provides interface icons and animation/motion effects.

Spreadsheet processing	xlsx (SheetJS)	Reads workbook data for import and spreadsheet-related handling.
Runtime / tooling	Node.js ecosystem, npm-compatible scripts, TypeScript compiler	Runs the development server and supports type-check/build scripts.
Persistence in current prototype	Browser localStorage	Keeps state in the current browser profile; it is not a shared server database.
Backend / integrations	No verified production database or live email/Calendly integration in the submitted prototype.	Communication, booking confirmation and virtual time actions are simulated or manually demonstrated in the current build.

Current limitations I would disclose

- The prototype is client-side and uses browser localStorage; records are not a shared multi-user source of truth.
- Email delivery, phone calls, calendar availability and Calendly booking events are simulated/demo flows, not verified live integrations.
- The even/odd row allocation is deterministic for demonstration; it is not a capacity-aware production assignment algorithm.
- The virtual operations clock exists for demonstrating time-based behaviour quickly; it should not be used for real-world timestamps.
- A production rollout would need authentication, role-based permissions, server-side storage, backups, audit retention, monitoring, privacy/security review and integration failure handling.

10. What I would do next

I would not move straight from a prototype to full automation. I would validate the rules with the people who run scheduling, then harden the parts where mistakes are most expensive.

Phase	Next step	Reason
1. Validate the workflow	Walk through real anonymized examples with an operations manager and both interns.	Check whether statuses, SLA thresholds and exception routes match reality.
2. Harden intake	Test the actual workbook formats, timestamp parsing, missing fields and duplicate matching.	Bad input can undermine every metric downstream.
3. Add shared persistence	Move from localStorage to a server-side database with authentication and audit history.	Multiple users need one reliable record and controlled access.
4. Connect external systems	Integrate approved email and Calendly APIs with retries, idempotency and webhook verification.	Remove manual steps without pretending that an attempted action is a completed booking.
5. Measure real operations	Track booking compliance, time to first outreach, exception rate and workload balance for a pilot.	Use evidence to adjust rules and staffing.
6. Scale responsibly	Load-test 500+ daily requests and assess permissions, privacy, monitoring and recovery.	A successful demo is not proof of production scale.

11. Closing note

The main thing I wanted to demonstrate with this assignment was how I approach an ambiguous operational problem. I started with the target and the people doing the work, broke the workflow into

smaller units, identified where spreadsheets and handoffs could fail, and designed a prototype around ownership, next action, SLA risk and verified outcomes.

My experience in HR analytics helped me recognise the practical cost of inconsistent data and manual tracking. I chose a control-tower approach because it makes the process easier to operate and easier to inspect. AI-assisted coding helped me turn that idea into a working prototype faster, but the product choices and the responsibility for explaining its assumptions and limitations remain mine.

I where to put this into a One-sentence summary
I turned a high-volume scheduling brief into a structured operating workflow, then used AI-assisted development to prototype a control tower that makes ownership, follow-ups, SLA risk and booking evidence visible.